Université Hassan 1er — FST Settat
Département Informatique
Année Universitaire : 2026–2027
Module : Cloud Computing, Virtualisation & DevOps
Enseignant : Pr. Abdelfattah Abassi

Série 6 — Docker Compose, bonnes pratiques et CI/CD

Durée : 2 h (TD : 40 min ; TP : 80 min)

Chapitre : Conteneurisation — Mise en pratique de Compose, sécurisation et automatisation

Cette séance prolonge les manipulations réalisées avec les conteneurs Docker. L'objectif est maintenant de faire fonctionner plusieurs services ensemble, puis d'observer ce qui se passe lorsque la base de données devient indisponible, lorsque les données doivent être conservées ou lorsqu'on modifie l'accès à l'API. Les exercices du TD servent à préparer les choix qui seront mis en uvre pendant le TP.

Objectifs de la série

À la fin de la séance, vous devrez être en mesure de :

- regrouper plusieurs commandes `docker run` dans un fichier `docker-compose.yml` ;
- lancer une petite application composée d'une API Flask et d'une base PostgreSQL, tout en conservant les données ;
- distinguer le démarrage d'un conteneur de la disponibilité effective du service qu'il héberge, notamment à l'aide d'un *healthcheck* ;
- examiner un Dockerfile et y apporter des améliorations concernant la taille de l'image, les étapes de construction, les droits et la gestion des secrets ;
- expliquer à quel moment Docker intervient dans un pipeline CI/CD et préciser ce que Compose ne remplace pas dans Kubernetes.

Partie TD (40 min)

Exercice 1 — De `docker run` à Compose (10 min)

À partir des commandes ci-dessous, rédiger un fichier `docker-compose.yml` qui décrit les deux services et les ressources nécessaires :

```
docker network create appnet
docker volume create pgdata
docker run -d --name db --network appnet \
-v pgdata:/var/lib/postgresql/data \
-e POSTGRES_PASSWORD=secret postgres:15
docker run -d --name api --network appnet -p 5000:5000 \
-e DATABASE_URL=postgresql://postgres:secret@db:5432/postgres monapi
:1.0
```

On considère que le Dockerfile de l'API se trouve dans le dossier courant. Indiquer comment demander à Compose de construire cette image. Faut-il également déclarer le réseau créé au début ? Justifier votre réponse.

Exercice 2 — Comprendre un fichier Compose (10 min)

On vous donne le fichier Compose suivant. Prenez quelques instants pour repérer les services, leurs images et les ressources qu'ils utilisent :

```
services:
api:
build: .
ports: ["5000:5000"]
depends_on: [db]
db:
image: postgres:15
volumes: ["data_db:/var/lib/postgresql/data"]
environment: ["POSTGRES_PASSWORD=secret"]
volumes:
data_db:
```

1. Différence entre `build:` et `image:` ?
2. Que garantit exactement `depends_on` ? Que ne garantit-il **pas** ? Quel risque cela crée-t-il pour l'API au démarrage ?
3. Quelle est la différence entre `docker compose down` et `docker compose down -v` ? Laquelle conserve les données de la base ?
4. Comment l'API joint-elle la base ?

Exercice 3 — Audit d'un Dockerfile (10 min)

```
FROM ubuntu:latest
RUN apt-get update
RUN apt-get install -y python3 python3-pip curl
COPY . /app
ENV DB_PASSWORD=admin123
WORKDIR /app
RUN pip3 install -r requirements.txt
CMD ["python3", "app.py"]
```

Relire ce Dockerfile comme si vous deviez le maintenir dans un projet réel. Relever au moins **six points à améliorer**, en précisant pour chacun le problème rencontré et la correction proposée. Pensez notamment à la taille de l'image, au cache de construction, à la sécurité et à la reproductibilité.

Exercice 4 — Multi-stage et CI/CD (10 min)

1. Expliquer le principe d'un *build* multi-stage et ses deux principaux avantages.
2. Un développeur vient de terminer une modification de son application. Replacer dans un ordre logique les opérations suivantes : *docker push*, *déploiement*, *push du code*, *tests automatisés* et *docker build*.
3. Pourquoi dit-on que Docker rend le déploiement reproductible ? Pourquoi éviter le tag `latest` pour les images déployées ?
4. Docker Compose peut-il remplacer Kubernetes pour une application de grande taille ? Justifier.

Partie TP (80 min)

Attention

Projet : `~/tp4/etudiants-api`. Avant de commencer, libérer les ports 5000 et 8080 (`docker ps -a`). Les paquets Python `Flask` et `psycopg2-binary` sont fournis dans le dossier `wheels/`.

Étape A — Construire l'application Flask + PostgreSQL (20 min)

requirements.txt

```
Flask==3.1.2
psycopg2-binary==2.9.10
```

app.py

```
import os
import time

import psycopg2
from flask import Flask, jsonify

app = Flask(__name__)
DSN = os.environ.get(
"DATABASE_URL", "postgresql://postgres:secret@db:5432/postgres")


def get_conn():
for _ in range(10):
try:
return psycopg2.connect(DSN)
except psycopg2.OperationalError:
time.sleep(1)
raise RuntimeError("Base indisponible")


@app.route("/")
def accueil():
return "API etudiants operationnelle"


@app.route("/init")
def init():
with get_conn() as conn, conn.cursor() as cur:
cur.execute("CREATE TABLE IF NOT EXISTS etudiants "
"(id SERIAL PRIMARY KEY, nom TEXT NOT NULL)")
return "Table creee"


@app.route("/ajouter/<nom>")
def ajouter(nom):
with get_conn() as conn, conn.cursor() as cur:
cur.execute("INSERT INTO etudiants (nom) VALUES (%s)", (nom,))
return nom + " ajoute"


@app.route("/etudiants")
def liste():
with get_conn() as conn, conn.cursor() as cur:
cur.execute("SELECT id, nom FROM etudiants ORDER BY id")
return jsonify(cur.fetchall())


if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Remarque

L'ajout par une requête `GET` (`/ajouter/<nom>`) est une simplification pédagogique pour tester avec `curl` ; une API REST réelle utiliserait `POST`.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY wheels /wheels
COPY requirements.txt .
RUN pip install --no-cache-dir --no-index --find-links=/wheels -r
requirements.txt
COPY app.py .
RUN adduser --disabled-password --gecos "" appuser
USER appuser
EXPOSE 5000
CMD ["python", "app.py"]
```

docker-compose.yml

```
services:
api:
build: .
ports:
- "5000:5000"
environment:
DATABASE_URL: postgresql://postgres:secret@db:5432/postgres
depends_on:
db:
condition: service_healthy
db:
image: postgres:15
environment:
POSTGRES_PASSWORD: secret
volumes:
- data_db:/var/lib/postgresql/data
healthcheck:
test: ["CMD-SHELL", "pg_isready -U postgres"]
interval: 3s
timeout: 3s
retries: 10
volumes:
data_db:
```

```
docker compose config
docker compose up -d --build
docker compose ps
curl http://localhost:5000/init
curl http://localhost:5000/ajouter/Aya
curl http://localhost:5000/ajouter/Youssef
curl http://localhost:5000/etudiants
docker compose logs -f api
```

À vérifier : observer la colonne d'état de `db` avec `docker compose ps`. À quel moment le statut `healthy` apparaît-il ? Comparer ensuite ce comportement avec le `depends_on: [db]` de l'exercice 2.

Étape B — Persistance des données (10 min)

```
docker compose down
docker compose up -d
curl http://localhost:5000/etudiants
docker compose down -v
docker compose up -d
curl http://localhost:5000/etudiants
curl http://localhost:5000/init
curl http://localhost:5000/etudiants
```

À observer : comparer le contenu de la table après `down` puis après `down -v`. Noter le message obtenu si vous interrogez `/etudiants` avant de rappeler `/init`, puis expliquer son origine.

Étape C — Panne de la base et tolérance de l'API (10 min)

```
docker compose stop db
curl -m 20 http://localhost:5000/etudiants
docker compose start db
curl http://localhost:5000/etudiants
docker compose logs api | tail -n 20
docker compose exec db psql -U postgres -c "SELECT count(*) FROM
etudiants;"
```

À observer : relever le résultat de la première requête lorsque PostgreSQL est arrêté. Après le redémarrage de la base, refaire le test. Expliquer le rôle de la boucle de tentatives présente dans `get_conn`.

Étape D — Ajouter un proxy Nginx (15 min)

Créer `nginx.conf` :

```
server {
    listen 80;
    location / {
        proxy_pass http://api:5000;
    }
}
```

Modifier `docker-compose.yml` : **supprimer** la section `ports` du service `api` et ajouter le service suivant sous `services:` :

```
proxy:
image: nginx:alpine
ports:
- "8080:80"
volumes:
- ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
depends_on:
- api
```

```
docker compose up -d
curl http://localhost:8080/etudiants
curl -m 3 http://localhost:5000/etudiants
docker compose ps
```

À expliquer : comparer l'accès à l'API par le port 8080 et par le port 5000. À partir de la configuration, expliquer le rôle de Nginx, le type de montage utilisé pour `nginx.conf` et l'intérêt de l'option `:ro`.

Étape E — Build multi-stage (15 min)

Dans un dossier `~/tp4/multistage`, créer deux Dockerfiles qui produisent la même page web ; l'étape de ń construction ż génère un gros fichier temporaire de 50 Mo, qui simule les outils de compilation.

Dockerfile.single

```
FROM nginx:alpine
RUN dd if=/dev/zero of=/tmp/gros_fichier bs=1M count=50 \
&& echo "<h1>Site statique</h1>" > /usr/share/nginx/html/index.html
```

Dockerfile.multi

```
FROM alpine AS build
RUN mkdir /site \
&& dd if=/dev/zero of=/tmp/gros_fichier bs=1M count=50 \
&& echo "<h1>Site statique</h1>" > /site/index.html

FROM nginx:alpine
COPY --from=build /site/index.html /usr/share/nginx/html/index.html
```

```
docker build -f Dockerfile.single -t site-single:1.0 .
docker build -f Dockerfile.multi -t site-multi:1.0 .
docker images | grep site-
docker run -d --name s1 -p 8083:80 site-multi:1.0
curl http://localhost:8083
docker run --rm site-multi:1.0 ls /tmp
docker run --rm site-single:1.0 ls -lh /tmp
```

À comparer : relever la taille des deux images et vérifier la présence du fichier temporaire dans chacune. En vous appuyant sur cette observation, expliquer l'intérêt du multi-stage dans un projet qui compile une application Node.js, Go ou Java.

Étape F — Sécurité et nettoyage (10 min)

```
docker compose exec api whoami
docker run --rm alpine whoami
docker compose exec api touch /etc/test
docker compose exec api sh -c 'echo $DATABASE_URL'
docker inspect $(docker compose ps -q api) | grep -i DATABASE_URL
docker rm -f s1
docker compose down -v
docker system df
```

À vérifier : identifier l'utilisateur de l'API, puis interpréter le résultat de la tentative d'écriture dans `/etc`. Enfin, examiner les informations affichées par `docker inspect` et en déduire les précautions à prendre pour protéger les véritables secrets.

À rendre en fin de séance

À la fin de la séance, déposer le dossier `etudiants-api` avec les fichiers `Dockerfile`, `app.py`, `requirements.txt`, `docker-compose.yml` et `nginx.conf`. Joindre également le tableau comparatif des tailles obtenu à l'étape E ainsi que les réponses argumentées aux questions des étapes A à F.

Annexe A — Aide-mémoire des commandes

Commande	Rôle
`docker run -d -p H:C -name N image`	Créer et démarrer un conteneur en arrière-plan, port hôte → conteneur
`docker ps` / `docker ps -a`	Lister les conteneurs actifs / tous les conteneurs
`docker stop|start|restart N`	Arrêter, démarrer, redémarrer un conteneur existant
`docker rm N` / `docker rm -f N`	Supprimer un conteneur arrêté / forcer la suppression
`docker images` / `docker rmi I`	Lister / supprimer des images
`docker exec -it N bash`	Ouvrir un shell dans un conteneur en cours
`docker logs -f N`	Suivre les journaux
`docker inspect N`	Détails JSON d'un conteneur, d'une image, d'un réseau
`docker build -t nom:tag .`	Construire une image depuis le Dockerfile du dossier
`docker history I`	Afficher les couches d'une image
`docker network create/ls/inspect/rm`	Gérer les réseaux
`docker volume create/ls/inspect/rm`	Gérer les volumes
`docker compose up -d -build`	Construire et démarrer tous les services
`docker compose ps / logs -f S / exec S cmd`	État, journaux, commande dans un service
`docker compose down [-v]`	Arrêter et supprimer (avec `-v` : et les volumes)
`docker system df`	Espace disque utilisé par Docker